# Reinforcement Learning-Based Computer Numerical Control Motion Planning with Adaptive Preview and Jerk-Constrained Action Projection

**Abstract**

Computer numerical control (CNC) motion planning converts tool paths generated by computer-aided manufacturing (CAM) into executable trajectories within contour and kinematic limits. Two-step methods fix the smoothed path before scheduling feedrate, limiting further adjustment of corner geometry within the tolerance band. We propose the Jerk-constrained Neural Numerical Controller (J-NNC) to coordinate trajectory shaping and feedrate regulation in a closed loop. The policy selects steering, feedrate and Adaptive Preview, while a kinematic constraint module (KCM) converts the combined request into feasible planar motion. The controller uses 14 current inputs without stored policy history. On a 44.420514 m closed path, it completed ordered traversal, braking and rest in 444.177 s. The planar velocity, acceleration and jerk limits were 100 mm/s, 2000 mm/s$^2$ and 62500 mm/s$^3$, respectively. Complete execution time was 1.008 s shorter than for a two-step planner. Speed deficit at every one of the 34 corners was no greater than for that planner or a trained two-action controller. On an 84-corner dense path, execution time was reduced by 25.65% relative to two-step planning and 6.07% relative to the fastest valid two-action result. The trajectories and motion profiles show turning before the corner, feedrate recovery and sustained near-limit cruising within the one-sided 1.25 mm Euclidean tolerance.

*Keywords:* computer numerical control, reinforcement learning, Adaptive Preview, jerk-constrained action projection

## 1. Introduction

Two-step computer numerical control (CNC) motion planning smooths the tool path and then schedules feedrate along the resulting curve [1, 2]. Smoothing improves the geometric continuity of paths generated by computer-aided manufacturing (CAM). Once the geometry is fixed, feedrate scheduling mainly adjusts speed along that curve [3, 4]. It has limited scope to adjust the trajectory within the contour-tolerance band, particularly near corners and curvature transitions [5, 6]. This separation makes it difficult to jointly optimize trajectory smoothness and feedrate under velocity, acceleration and jerk limits [1, 7].

Conventional planning combines local or global path smoothing with feedrate scheduling. Local smoothing constructs transition curves between adjacent blocks. Its limited computational scope and explicit contour-error bounds suit real-time interpolation [8]. Higher-order Bézier curves, B-splines and curvature-smooth transitions can improve acceleration or jerk continuity near corners [9, 10]. Dense short segments and consecutive corners often limit the available transition length [2]. Global smoothing reconstructs several segments together to improve continuity along dense short-block paths [11]. Under

strict contour-error bounds, it must balance computational cost, iterative fitting, local feature preservation and real-time execution [12]. Feedrate scheduling imposes velocity, acceleration and jerk limits along the selected geometric path [7]. Non-uniform rational B-spline (NURBS) interpolation and S-shaped profiles improve feasibility under chord-error and axis-drive constraints [13]. Look-ahead interpolation uses upcoming geometry to regulate feedrate along short-line and spline paths [14, 15].

Path geometry determines the curvature distribution and therefore affects the admissible feedrate. In a serial procedure, however, the feedrate scheduler cannot readily revise the transition geometry selected during smoothing. This restriction is particularly relevant when short segments, consecutive corners and rapid curvature changes occur along the same path [2, 16]. A conservative transition may introduce unnecessary deceleration, while a more aggressive transition may still require slowing down to satisfy acceleration and jerk limits [5, 6]. Jointly selecting geometry and feedrate could therefore make better use of the tolerance band under the same motion limits [1, 7].

One-step planning methods address this coupling by combining trajectory smoothing and feedrate planning. Direct axis-velocity blending and integrated acceleration and deceleration interpolation plan corner motion under real-time constraints [6, 17]. Minimum-time cornering and high-speed cornering with vibration suppression optimize traversal under contour-error and actuator limits [5, 18]. Driving-force-constrained smoothing accounts for drive capability, while quadratic B-spline fitting combines path construction and optimal velocity interpolation under bounded error [12, 19]. Integrated methods for line–arc and five-axis paths also address transition overlap and feedrate fluctuations [20, 21]. Li et al. investigated reinforcement learning for CNC trajectory smoothing [22], providing a basis for learning decisions that couple local trajectory shaping with feedrate regulation.

Planning within a tolerance band also requires the selected geometry and feedrate to remain feasible throughout execution. Many optimization methods require predefined transition curves, local corner models, nonlinear constraints or iterative computation. Their performance can depend on segment length, corner distribution and local geometry [12, 20]. In learned one-step planning, contour tracking and path progress must also satisfy explicit velocity, acceleration and jerk limits during execution. Steering, feedrate and preview distance jointly affect corner transitions. The resulting motion must therefore be compared using executed trajectories and complete traversal times under common contour and kinematic limits.

This paper proposes the Jerk-constrained Neural Numerical Controller (J-NNC) for joint CNC motion planning within a contour-tolerance band. The policy jointly selects steering, feedrate and Adaptive Preview from current geometric and kinematic observations. A kinematic constraint module (KCM) converts the combined request into interpolation commands that satisfy velocity, acceleration and jerk limits. Policy learning uses proximal policy optimization (PPO) [23], with an objective based on execution time and path progress under contour and motion constraints. Fig. 1 compares the conventional serial approach with the J-NNC framework.

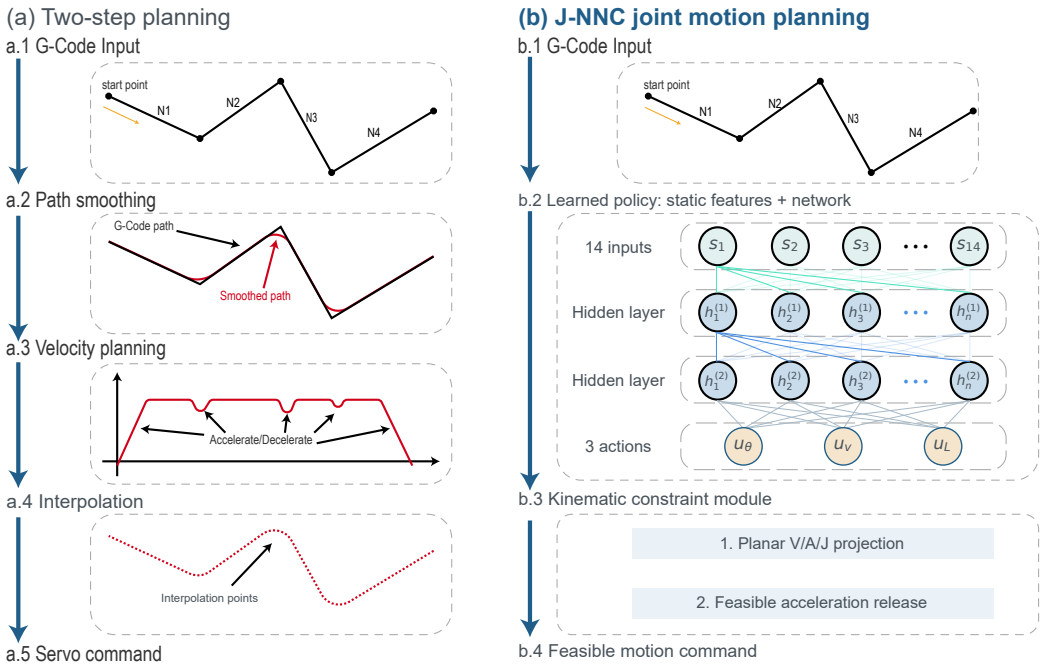



Fig. 1: Conventional two-step planning and J-NNC joint motion planning. The policy uses 14 current inputs to select three actions. The KCM converts their combined request into feasible planar motion. In the diagram, $N_i$ labels a path block, $s_i$ is an input component, and $h_i^{(j)}$ is the output of unit $i$ in hidden layer $j$, with $n$ denoting the last unit. The outputs $u_\theta, u_v, u_L$ request steering, feedrate and preview, respectively. V/A/J denotes velocity, acceleration and jerk.

The main contributions are as follows:

(1) A one-step formulation jointly selects steering, feedrate and Adaptive Preview to coordinate trajectory shaping and feedrate regulation within the tolerance band.

(2) A KCM projects the combined action request into executable commands that satisfy planar vector velocity, acceleration and jerk limits.

(3) Complete executions on a long closed path and a corner-dense path show lower speed deficit and shorter traversal times than two-step planning and a trained two-action controller.

Section 2 defines the planning problem and its constraints. Section 3 presents the method, and Section 4 reports the experimental settings, main results and comparison with fixed-preview control. Section 5 concludes the paper.

## 2. Problem Formulation

The planning problem is defined by the reference path, contour tolerance and constraints on the executed motion.

### *2.1. Tool Path and Geometric Feasible Region*

Consider a discrete sequence of cutter-location points generated by a CAM system,

$$P_{\text{raw}} = \{p_1, p_2, \ldots, p_N\}, \tag{1}$$

Here $P_{\text{raw}}$ is the reference path, $N$ is the number of cutter-location points, and $p_i \in \mathbb{R}^2$ is point $i$ in the two-dimensional interpolation plane. The prescribed line and arc segments connect these points. Its Cartesian axes are denoted by $X$ and $Y$. Let $\varepsilon$ denote the full width of the contour-tolerance band. Offsetting the reference path by $\varepsilon/2$ on each side defines the geometric feasible region $\Omega$ for a position $x$:

$$\Omega = \{x \mid d(x, P_{\text{raw}}) \leq \varepsilon/2\}, \tag{2}$$

where $d(x, P_{\text{raw}})$ is the minimum Euclidean distance from $x$ to the finite reference path, including its line and arc segments. The one-sided tolerance is therefore $\varepsilon/2$. Within this band, the controller can adjust the local trajectory to smooth corner transitions while maintaining contour accuracy.

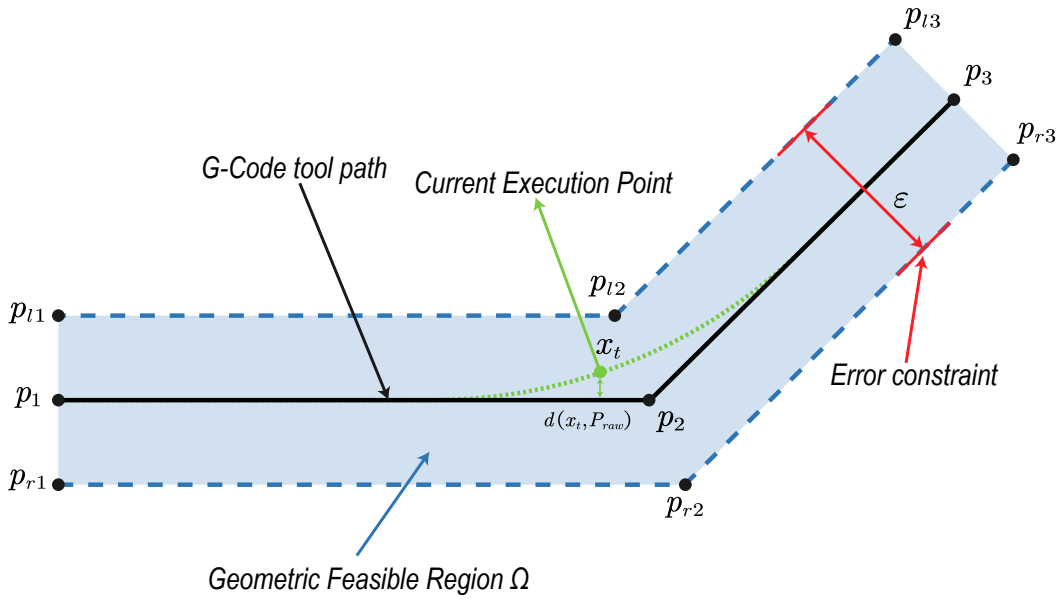



Fig. 2: Geometric feasible region formed by the reference path and the contour tolerance band. Points $p_{li}$ and $p_{ri}$ mark the left and right boundaries associated with reference point $p_i$; $x_t$ is the current execution point at decision index $t$.

The reference path is parameterized by arc length for closed-loop control. Tracking variables and forward geometry are extracted at the current projected arc length $\ell_t$ at decision index $t$ to form the observation. An ordered sequence of gates preserves traversal order where different portions of the closed path lie close together.

### *2.2. Constrained Action Decision Formulation*

At each decision interval $\Delta t$, the policy receives the current state $s_t$ and returns an action $u_t$. The execution layer converts this request into feasible planar motion at interval $h$. During training, the environment updates the position and motion variables and returns reward $r_t$. For policy $\pi$, discount factor $\gamma \in [0, 1]$ and final decision index $H$, the constrained decision problem is

$$\max_{\pi} \ \mathbb{E}_{\pi}\left[\sum_{t=0}^{H} \gamma^t r_t\right], \tag{3}$$

where $\mathbb{E}_{\pi}$ denotes expectation over trajectories generated by $\pi$. At execution index $k$, the position is $x_k$ and the planar velocity, acceleration and jerk vectors are $v_k, a_k, j_k$. Their

magnitude limits are $V_{\max}, A_{\max}, J_{\max}$, respectively, and $\|\cdot\|_2$ denotes the Euclidean norm. The constraints are $x_k \in \Omega$ and

$$\|v_k\|_2 \le V_{\max}, \qquad \|a_k\|_2 \le A_{\max}, \qquad \|j_k\|_2 \le J_{\max}. \tag{4}$$

These bounds apply to planar vectors and include the acceleration and jerk associated with turning. The motion variables are reconstructed from executed positions as

$$v_k = \frac{x_k - x_{k-1}}{h}, \quad a_k = \frac{v_k - v_{k-1}}{h}, \quad j_k = \frac{a_k - a_{k-1}}{h}. \tag{5}$$

The initial velocity and acceleration are zero. Derivatives of scalar speed omit changes in direction and cannot fully assess these vector constraints. Heading and curvature describe the resulting trajectory; no separate angular-axis limits are imposed.

A successful execution traverses the entire reference path in order and keeps every command interval within the Euclidean tolerance band. It also completes braking and remains at rest for at least 20 ms. Complete execution time includes startup, traversal, braking and this rest period, so the comparison covers the same complete task.

## 3. Methodology

### *3.1. Overall Framework*

Adaptive Preview is the preview distance selected by the policy from the current state.

Motion planning for mixed tool paths is formulated as a closed-loop reinforcement learning problem. The policy acts in a path-simulation environment with prescribed contour and kinematic limits. At each decision cycle, the environment constructs the state from tracking error, motion variables and forward geometry. The policy supplies steering, feedrate and Adaptive Preview requests, which the KCM converts into executable planar motion. The executed motion updates the state and provides reward feedback during training.

Fig. 3 shows the control loop. Preview selects the forward guide target, steering corrects its direction, and feedrate sets the requested speed. The execution layer constrains the next velocity using the current velocity and acceleration vectors. Evaluation uses the same feedback loop with fixed policy parameters. In the diagram, $f_\theta$ maps the current state $s$ to action $u$, and $\theta$ denotes its fitted parameters. The decoded requests $\alpha, F, L$ are steering correction, feedrate and preview distance. Subscripts are omitted for the held state and action; $v_{k+1}$ is the next executed velocity.

### *3.2. State Design*

The controller uses a 14-dimensional observation of the current state. Tracking and motion variables describe the present motion, while near and far reference points describe upcoming geometry. The columns of $R_h$ are the current heading unit vector and its left normal. Let $t_r, n_r$ denote the local reference unit tangent and unit normal, and let $p(\ell)$ denote a reference point. The current position, velocity and acceleration vectors are $x, v, a$, with time indices omitted. Here $\ell$ is the projected arc length, $d$ is a forward sampling distance, and $\theta_r, \theta_h$ are the reference-tangent and current-heading angles. The input is

$$\begin{aligned} s = [&e_n/1.25,\ R_h^{\mathsf{T}} v/100,\ R_h^{\mathsf{T}} a/2000,\ q_{30}^{\parallel},\ \sin(\theta_r - \theta_h),\\ &\{R_h^{\mathsf{T}}[p(\ell + d) - x]/15\}_{d\in\{2,6,15\}},\ q_{30}^{\perp}], \end{aligned} \tag{6}$$

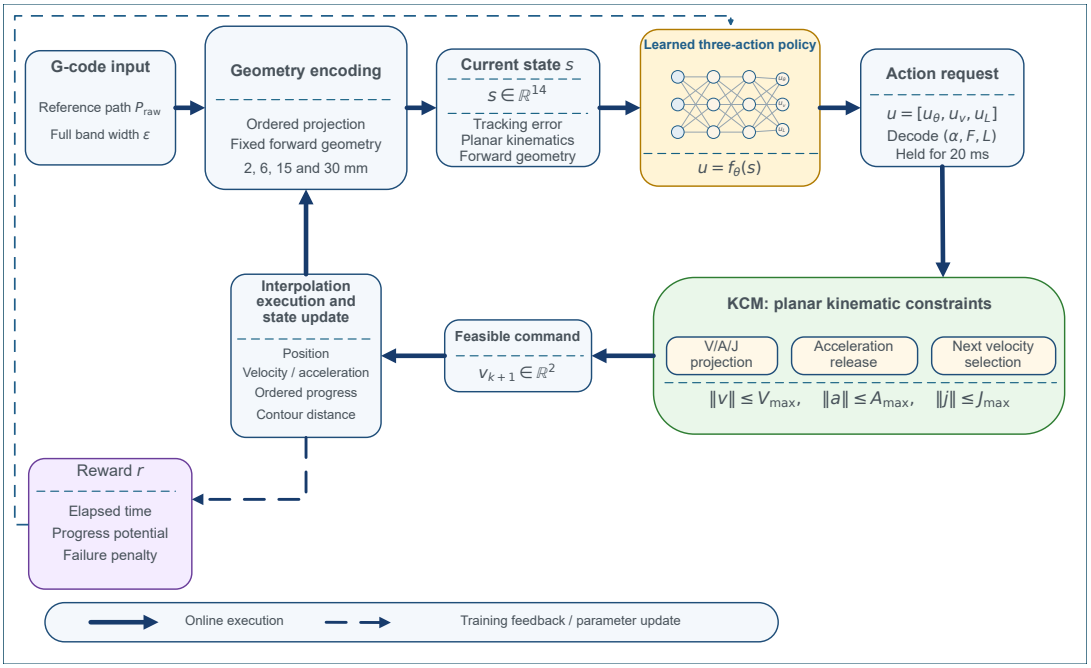



Fig. 3: J-NNC control loop. Forward geometry sampled at fixed distances and current motion variables form the 14-dimensional state. The policy outputs steering, feedrate and Adaptive Preview requests, and the KCM determines feasible planar commands. The dashed outer connection represents training feedback and parameter updates.

where $e_n = [x - p(\ell)] \cdot n_r$ is the signed normal tracking error. The dimensionless far-reference components $q_{30}^{\parallel}$ and $q_{30}^{\perp}$ are the tangential and normal projections of the displacement to the point 30 mm ahead:

$$q_{30}^{\parallel} = \frac{[p(\ell + 30) - p(\ell)] \cdot t_r}{30}, \qquad q_{30}^{\perp} = \frac{[p(\ell + 30) - p(\ell)] \cdot n_r}{30}. \tag{7}$$

Lengths are expressed in millimeters. The two far-reference components are expressed in the reference frame. Velocity, acceleration and the three near-reference point pairs are expressed in the heading frame. The 14 inputs comprise one tracking error, four kinematic components, one heading sine, six near-point coordinates and two far-reference components.

The near-reference points describe the local transition and recovery, while the 30 mm displacement provides information about geometry farther ahead. These fixed sampling distances are distinct from the preview distance selected by the action. Blending adjacent estimates of reference progress reduces observation jumps at segment boundaries without changing the ordered coverage sequence. The observation excludes elapsed time, global coordinates, corner identifiers, remaining path length and previous actions. The policy stores neither a recurrent state nor an observation history. The static nonlinear features in Appendix B depend only on these 14 current inputs.

### *3.3. Action Space and Kinematic Constraint Module*

At decision $t$, the action $u_t$ contains the normalized steering, feedrate and preview components $u_{\theta,t}, u_{v,t}, u_{L,t}$, each clipped to $[-1, 1]$:

$$u_t = [u_{\theta,t}, u_{v,t}, u_{L,t}]. \tag{8}$$

The normalized outputs are mapped to physical requests as

$$\alpha_t = \frac{\pi}{2} u_{\theta,t}, \qquad F_t = 50(1 + u_{v,t}), \qquad L_t = 6.4 + 5.6 u_{L,t}. \tag{9}$$

Here $\alpha_t$ is a steering correction in radians, $F_t$ is the requested feedrate in mm/s, and $L_t \in [0.8, 12]$ mm is the Adaptive Preview distance. The steering action corrects direction rather than commanding angular velocity. The three requests are held for 20 ms, while the guide direction and motion are updated at the smaller integration interval $h$.

The guide direction points from the current position toward the reference point selected by preview distance $L$. Steering corrects this direction, and filtering smooths its changes between interpolation steps. The filtered direction and feedrate $F$ define the desired velocity. Using velocity and acceleration feedback, the execution layer forms a next-velocity request for KCM projection. A release check verifies that acceleration can subsequently decrease within the constraints (Appendix A). Contour tolerance and complete ordered coverage are checked separately.

### *3.4. Reward Design*

The learning objective combines elapsed execution time, progress shaping and a failure penalty:

$$r_t = -\Delta t_t + \Phi_{t+1} - \Phi_t - \mathbf{1}_{\text{failure}} P_t, \qquad \Phi_t = \frac{\ell_t - \ell_{\text{total}}}{V_\Phi}. \tag{10}$$

Here $\Delta t_t$ is the actual duration of decision step $t$, $\Phi_t$ is a progress potential, $\ell_{\text{total}}$ is total reference length, and $V_\Phi$ scales distance to time. The indicator $\mathbf{1}_{\text{failure}}$ is one on failure and zero otherwise; $P_t$ is the failure penalty. The last PPO training stage uses $V_\Phi = 100$ mm/s and zero terminal potential. Failure incurs $P_t = 5 + \ell_{\text{rem}}/100$ seconds, where $\ell_{\text{rem}}$ is the remaining reference distance in millimeters. For successful undiscounted episodes with the same initial state, the potential differences sum to a constant. Maximizing the return therefore minimizes complete execution time. The learning procedure is described in Appendix B. Performance is evaluated using executed motion and completion time rather than reward values.

Policy learning combines PPO [23] with fitting to recorded feasible actions. The fixed policy combines a neural network with static features of the same 14 inputs; its complete expression and learning settings appear in Appendix B.

## 4. Experiments and Results

### *4.1. Experimental Settings*

The experiments assess motion feasibility and the effect of joint trajectory shaping and feedrate regulation on complete execution time. The long closed path has a reference length of 44.420514 m, and all 34 corners are included in the analysis. Figure 4 identifies its longest and shortest continuous straight sections (2262.742 and 80 mm) and extreme included angles (45.000° and 175.679°). It also marks two 90° arcs with radii of 600 and 800 mm. Continuous straight lengths are measured after merging adjacent collinear blocks with the same direction, including blocks spanning the closed-path junction. The enlarged view of the shortest straight section uses a 10 mm Cartesian grid. A second closed path is 1438.009 mm long and contains 84 corners (Fig. 9a). The long path assesses complete traversal, all-corner performance and sustained cruising; the dense path tests

efficiency during consecutive transitions. Both paths were used for policy development and model selection.

The primary results use a 1 ms interpolation interval. The full tolerance-band width is $\varepsilon = 2.5$ mm, corresponding to a one-sided Euclidean tolerance of 1.25 mm. All methods use the same planar vector limits: $V_{\max} = 100$ mm/s, $A_{\max} = 2000$ mm/s$^2$ and $J_{\max} = 62500$ mm/s$^3$. The learned constrained controllers update their action requests every 20 ms. Successful execution requires complete ordered traversal, geometric feasibility throughout each command interval, full braking and at least 20 ms of rest. The long-path evaluation includes all 2362 ordered gates and the final segment. A supplementary evaluation at 0.5 ms checks that the time gains exceed the effects of execution resolution (Appendix C).

The Neural Numerical Controller (NNC) comparison adapts the original direct-action checkpoint based on learned CNC smoothing [22]. The evaluated controller uses 14 inputs and 1 ms decisions, removes angular-motion feedrate caps and uses no KCM. Position commands are recorded from rest for up to 1 s or until termination. Figure 5 evaluates this adaptation under the present constraints. The two-step planner schedules jerk-limited motion after quintic Bézier smoothing adapted from Sencer et al. [24]; circular arcs retain their analytical form. No method receives a global feedrate reduction.

The two-action comparison retains steering and feedrate, with the same 14 inputs, execution layer and number of joint-action features (Appendix B). It receives separate fitting, parameter selection and PPO training. The selected controller completes both cases. Dense-case comparisons also retain the fastest valid two-action result, including models that fail on the long path.

For long-route corner $c$, let $\ell_c$ be its reference arc length and $\ell_k$ the ordered reference progress at execution step $k$. Integrated speed deficit $D_c$ is

$$D_c = h \sum_{k:\,|\ell_k - \ell_c| \le 10 \text{ mm}} \max(100 - \|v_k\|_2, 0). \tag{11}$$

This distance-valued metric captures both the magnitude and duration of speed reduction. All 34 complete corner windows are included, with numerical ties allowed within $10^{-7}$ mm. Near-limit cruising is defined throughout as $\|v_k\|_2 \ge 99$ mm/s. The utilization curves are $\|v\|_2/V_{\max}$, $\|a\|_2/A_{\max}$ and $\|j\|_2/J_{\max}$, reconstructed from executed planar positions using Eq. (5). A value of one indicates the corresponding limit; the vector norms include the motion associated with changes in direction.

### *4.2. Main Comparative Results*

J-NNC completed both paths within the common contour and planar kinematic limits (Tab. 1). Its complete execution time on the long path was 444.177 s, saving 1.008 s over two-step planning and 0.537 s over the selected two-action controller. On the dense path, J-NNC took 14.839 s, compared with 19.957 s for two-step planning, a reduction of 25.65%. These times include startup, every corner transition, braking and terminal rest.

Figure 5 shows repeated constraint violations by the adapted NNC during the first 80 ms, including after startup. In the retained first-second records, peak acceleration was 20.70/29.54 times the limit on the long/dense paths; peak jerk was 841.69/1150.81 times the limit. J-NNC remained within the bounds. This adaptation therefore has no feasible complete execution time under the prescribed limits.

Figure 6 compares the methods at the selected 90° corner C02 using common axes. J-NNC maintained a higher cornering speed within the same acceleration and jerk limits.

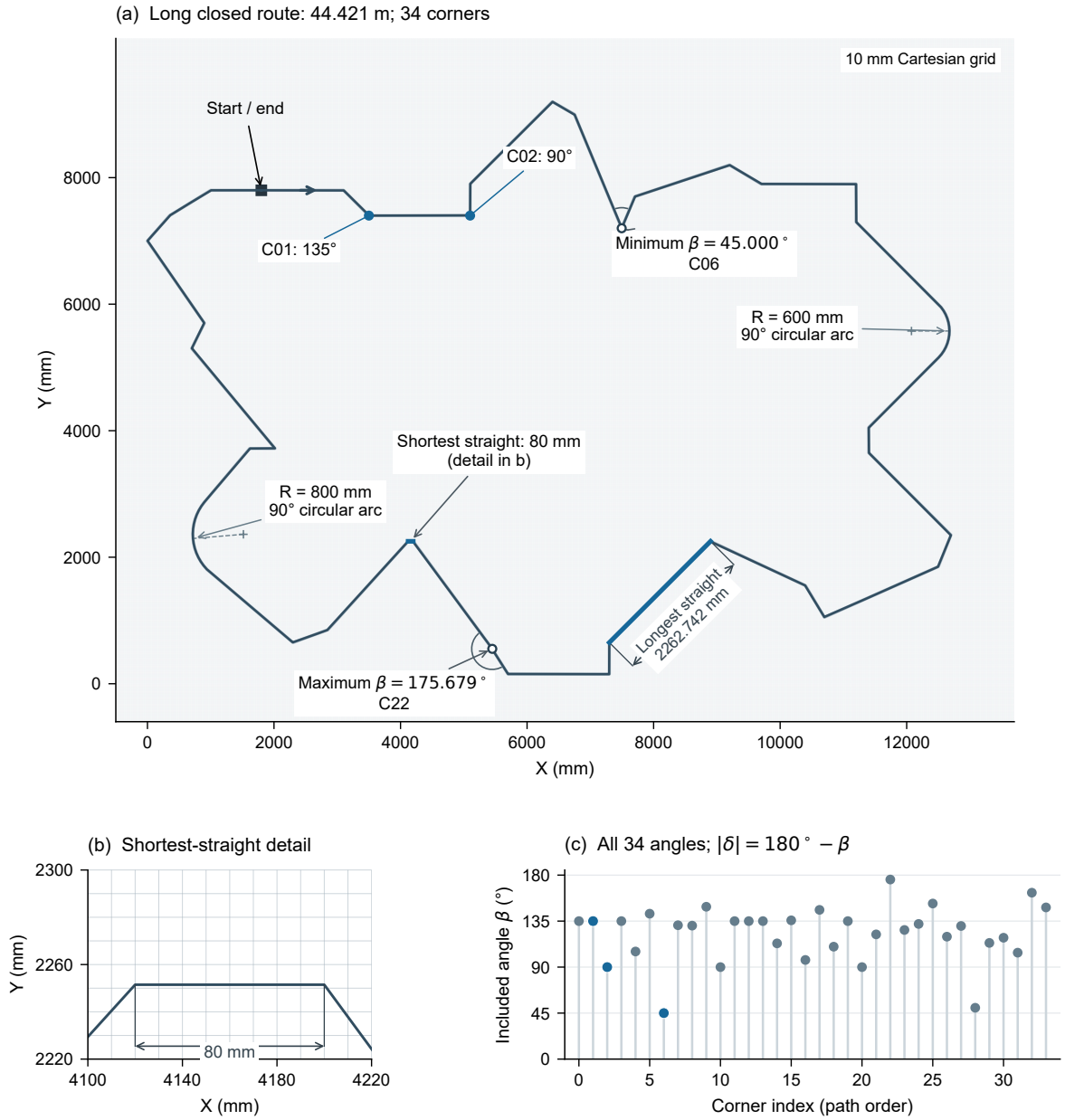



Fig. 4: Long-path geometry and corner angles: (a) complete path with the longest and shortest continuous straight sections, extreme included angles and circular arcs; (b) enlarged shortest straight section on a 10 mm grid; (c) all 34 included angles. Here $\beta$ is the included corner angle, $\delta$ is the signed direction change, and $R$ is arc radius. C00–C33 identify corners in path order; C01, C02 and C06 are used in the local comparisons.

Integrated speed deficit, which includes the magnitude and duration of deceleration, was 86.1% lower than for two-step planning and 36.7% lower than for two-action control. Complete V/A/J profiles, including terminal braking and rest, are provided in Fig. C.1.

Table 2 summarizes utilization over the complete executions. On the dense path, mean feedrate utilization was 67.139% for two-step planning, 86.528% for two-action control and 91.071% for J-NNC. Both learned controllers had 95th-percentile acceleration and jerk utilization of 99.900%. J-NNC achieved a higher mean feedrate under the same limits. The low acceleration and jerk percentiles on the long path reflect the large proportion of cruising. Figure 6 shows utilization during the local corner transition.

Tab. 1: Feasibility and complete execution time at a 1 ms interpolation interval. Successful runs satisfy ordered traversal, contour tolerance throughout each command interval, planar V/A/J limits and terminal rest.

| Case | Method | Feasible | Complete time (s) | Maximum contour deviation (mm) |
|---|---|---|---|---|
| Long path | **J-NNC** | Yes | **444.177** | 1.221862 |
| | Two-action | Yes | 444.714 | 1.095198 |
| | Two-step planner | Yes | 445.185 | 1.243758 |
| | NNC | No | — | — |
| Dense case | **J-NNC** | Yes | **14.839** | 1.154901 |
| | Two-action | Yes | 15.875 | 1.121741 |
| | Two-step planner | Yes | 19.957 | 1.149675 |
| | NNC | No | — | — |

Maximum contour deviation is evaluated throughout each command interval. NNC violates the acceleration and jerk limits; dashes indicate that no feasible complete result is available.

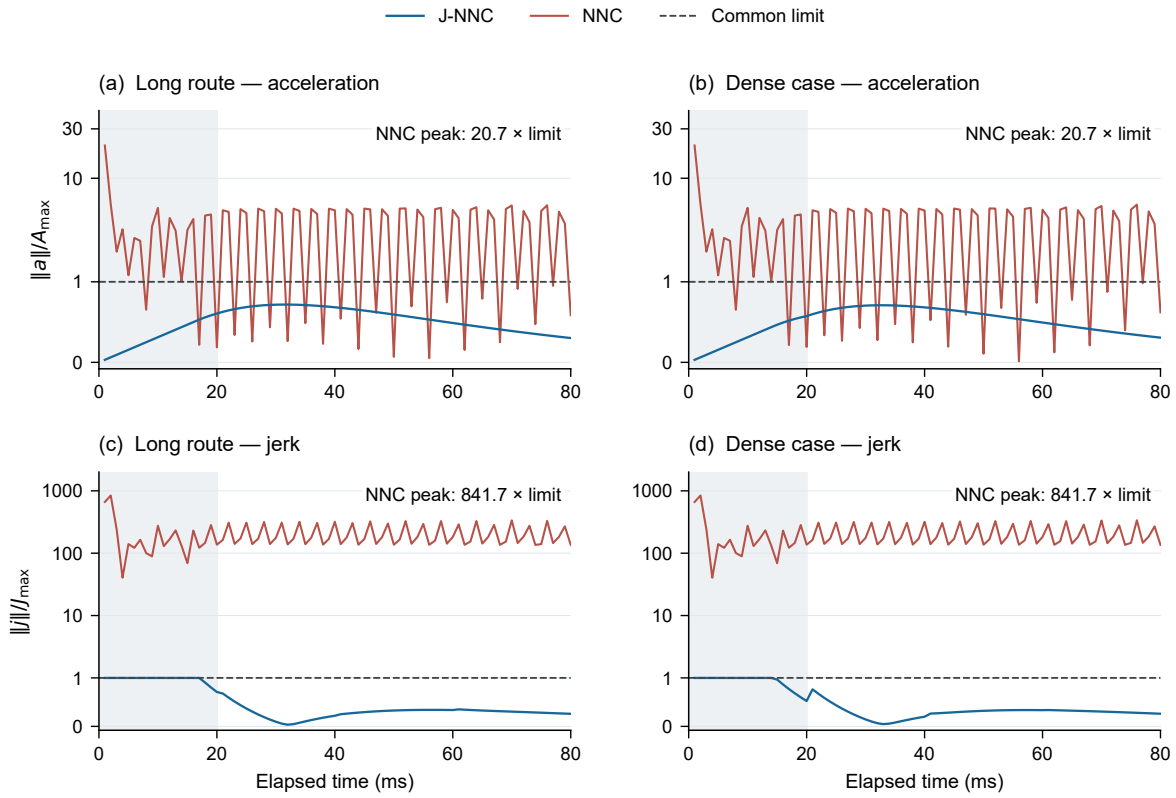



Fig. 5: Constraint violations of the adapted NNC at 1 ms: normalized vector acceleration (a,b) and jerk (c,d) during the first 80 ms of each case. Shading marks the initial 20 ms, and dashed lines indicate the common limits. The vertical axes are linear from zero to one and logarithmic above one. Peak annotations apply only to the displayed interval. Curves are reconstructed from recorded planar positions.

Tab. 2: Kinematic-limit utilization over complete executions at 1 ms. Values are expressed as percentages of the common planar vector limits.

| Case | Method | Mean $V$ utilization | 95th-percentile $A$ utilization | 95th-percentile $J$ utilization |
|---|---|---|---|---|
| Long path | **J-NNC** | **99.939** | 0.678 | 0.370 |
| | Two-action | 99.870 | 0.731 | 0.536 |
| | Two-step planner | 99.716 | 0.625 | 0.004 |
| Dense case | **J-NNC** | **91.071** | 99.900 | 99.900 |
| | Two-action | 86.528 | 99.900 | 99.900 |
| | Two-step planner | 67.139 | 67.696 | 98.084 |

Statistics include all equally spaced samples during startup, traversal, braking and rest. The 95th percentiles describe the distribution over the full execution and do not represent peak utilization.

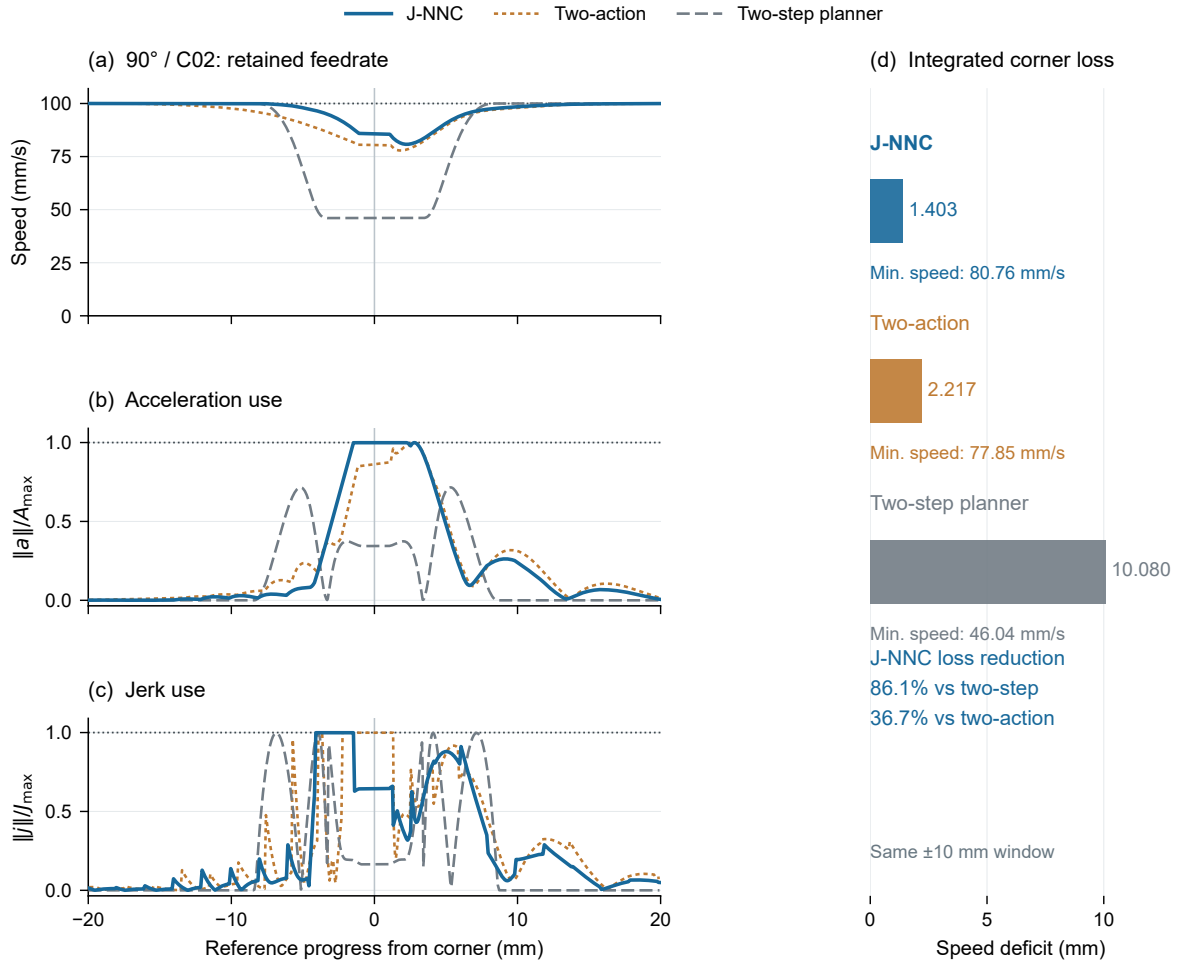



Fig. 6: Motion at the selected 90°/C02 corner from the same 1 ms records. Panels (a)–(c) align speed, normalized vector acceleration and normalized vector jerk by reference progress; dotted horizontal lines indicate the common limits. Panel (d) gives integrated speed deficit and minimum speed within the common ±10 mm window. The profiles span ±20 mm to include approach and recovery.

Figure 7 compares the executed trajectories at the selected 135°, 90° and 45° corners. The upper row uses local coordinates $x$ along the incoming tangent and $y$ along the inward normal. These plotting coordinates are distinct from the position vector $x$ in the motion equations. The lower row enlarges the marked transition regions. Equal aspect ratios and common scales within each row allow direct comparison of the trajectory shapes. J-NNC began turning inward before the vertex and spread the direction change through the tolerance band, consistent with its higher cornering feedrate in Fig. 6.

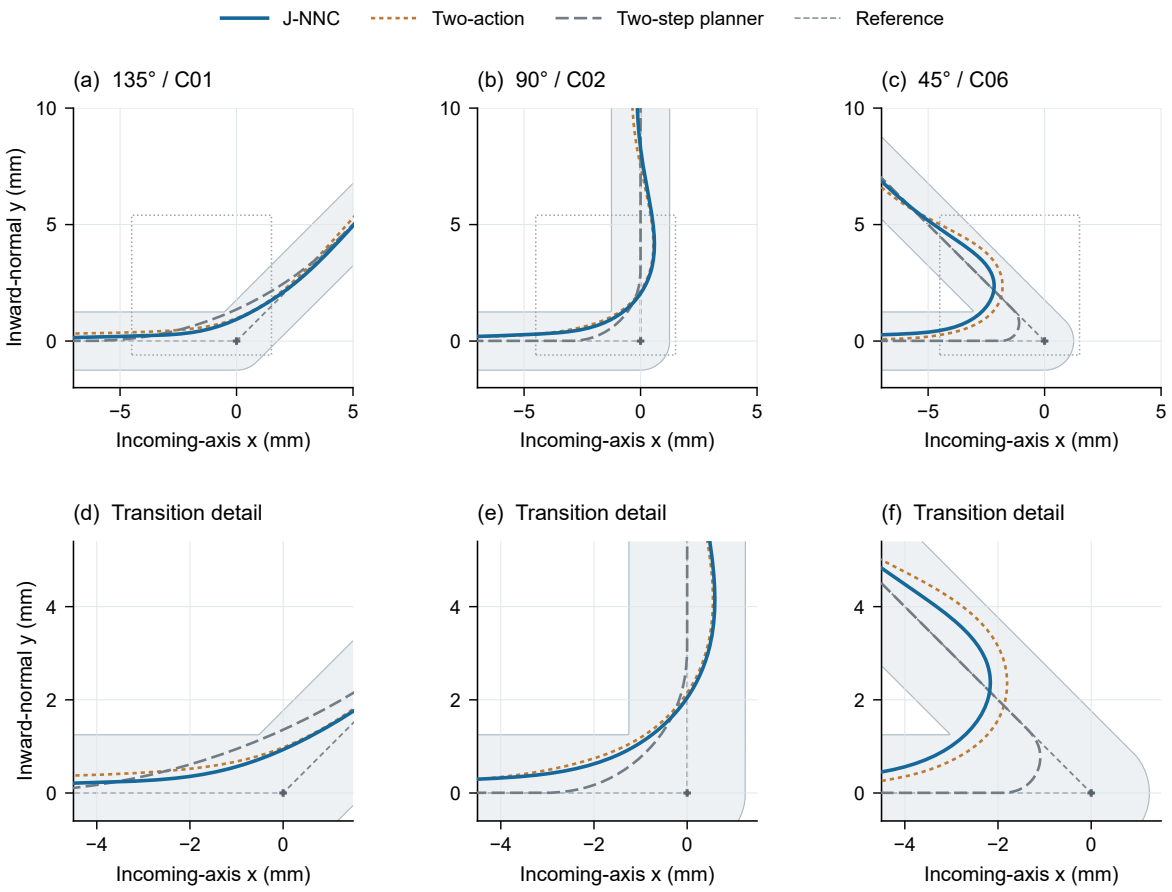


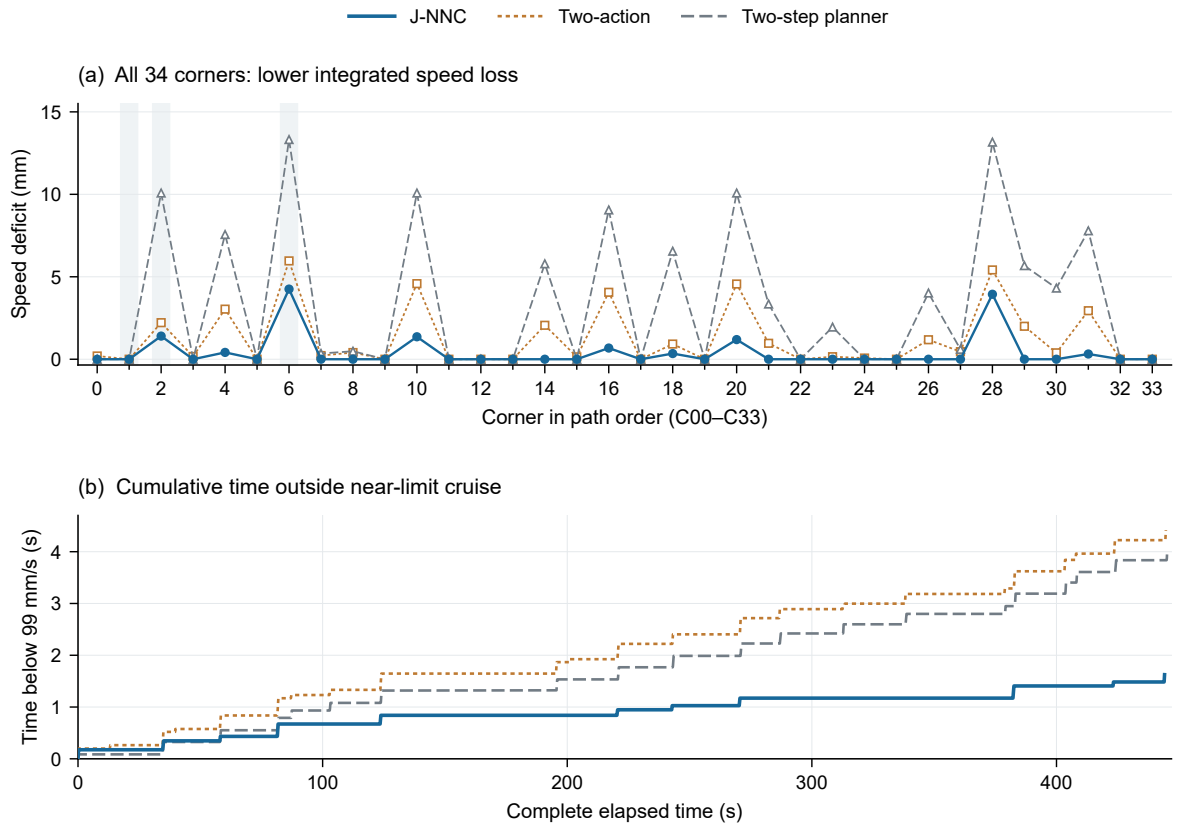

Fig. 7: Executed trajectories at the selected 135°/C01, 90°/C02 and 45°/C06 corners. Panels (a)–(c) show approach and recovery; dotted rectangles mark the enlarged regions in (d)–(f). Shading indicates the 1.25 mm Euclidean tolerance band. Each row uses equal aspect ratios and common scales. The trajectories are extracted from the 1 ms executions of the corresponding controllers reported in Table 1.

J-NNC traversed the 135° corner at 100 mm/s. At the 90° corner C02, its minimum speed was 80.76 mm/s, compared with 46.04 mm/s for two-step planning and 77.85 mm/s for two-action control. Integrated speed deficit was 10.080 mm for the two-step planner, 2.217 mm for the two-action controller and 1.403 mm for J-NNC. At the sharper C06 corner, the corresponding deficits were 13.321, 5.963 and 4.253 mm. J-NNC began turning inward before the vertex at all 34 long-path corners and all 84 dense-path corners.

The trajectory shapes and utilization profiles illustrate the coupling between geometry and feedrate. For a smooth trajectory, the signed normal acceleration $a_n$ satisfies $a_n = \|v\|_2^2 \kappa$, where $\kappa$ is signed trajectory curvature; jerk during turning also depends on curvature variation along the trajectory. Steering and preview distribute the direction change, and feedrate adjusts the acceleration and jerk required to follow it. The displayed trajectories use limited outward recovery within the tolerance band and achieve

lower speed deficit. Such outward displacement can provide additional turning space within the band; it is not directly rewarded.

Figure 8 reports the results for every corner on the long path. J-NNC had no greater integrated speed deficit than the two-step planner or trained two-action controller at any of the 34 corners, allowing numerical ties. Summed corner deficit decreased from 114.203 mm for two-step planning and 42.107 mm for two-action control to 13.902 mm for J-NNC. The cumulative full-path curves remain flat during near-limit cruising and show where additional deceleration occurs in the second half of the lap.




| Method | ≥99 mm/s time (s) | Time share (%) | Longest interval (s) |
|---|---|---|---|
| **J-NNC** | **442.552** | **99.634** | **111.753** |
| Two-action | 440.299 | 99.007 | 71.524 |
| Two-step planner | 441.238 | 99.113 | 71.569 |

Fig. 8: Corner speed deficits and full-path cruising at 1 ms: (a) integrated speed deficit in each ±10 mm corner window, with representative corners shaded; (b) cumulative time below 99 mm/s. The inset table gives time at or above 99 mm/s, its fraction of complete execution time and the longest continuous interval. All 34 corners and terminal rest are included.

J-NNC spent 442.552 s at or above 99 mm/s, compared with 441.238 s for two-step planning and 440.299 s for two-action control. The longest continuous cruising intervals were 111.753, 71.569 and 71.524 s, respectively. Lower corner speed deficit coincided with longer sustained cruising and shorter complete execution time.

To distinguish path shortening from speed effects, let $L_{\mathrm{exec}} = h \sum_k \|v_k\|_2$ be executed path length and $D_{\mathrm{all}} = h \sum_k (100 - \|v_k\|_2)$ be the full-path integrated speed deficit. Both sums cover all execution steps, including rest. Complete execution time $T$ then satisfies,

up to floating-point roundoff,

$$T = \frac{L_{\text{exec}} + D_{\text{all}}}{100}. \tag{12}$$

The terminal rest period is included in $D_{\text{all}}$. J-NNC traveled 44390.406 mm with a total speed deficit of 27.294 mm; the two-step planner traveled 44392.151 mm with a deficit of 126.349 mm. Of the 1.008 s saved, approximately 0.991 s came from lower speed deficit and 0.017 s from shorter travel. Against the two-action controller, the 0.537 s saving comprised 0.303 s from lower speed deficit and 0.234 s from shorter travel. The dense case examines performance when successive transitions occupy a larger fraction of the path.

### *4.3. Comparison with Fixed-Preview Control*

J-NNC completed the dense path in 14.839 s, compared with 15.875 s for the selected two-action controller. This corresponds to a 6.53% reduction under the same physical constraints. Against the fastest valid two-action result among all evaluated models, 15.798 s, J-NNC saved 0.959 s (6.07%).

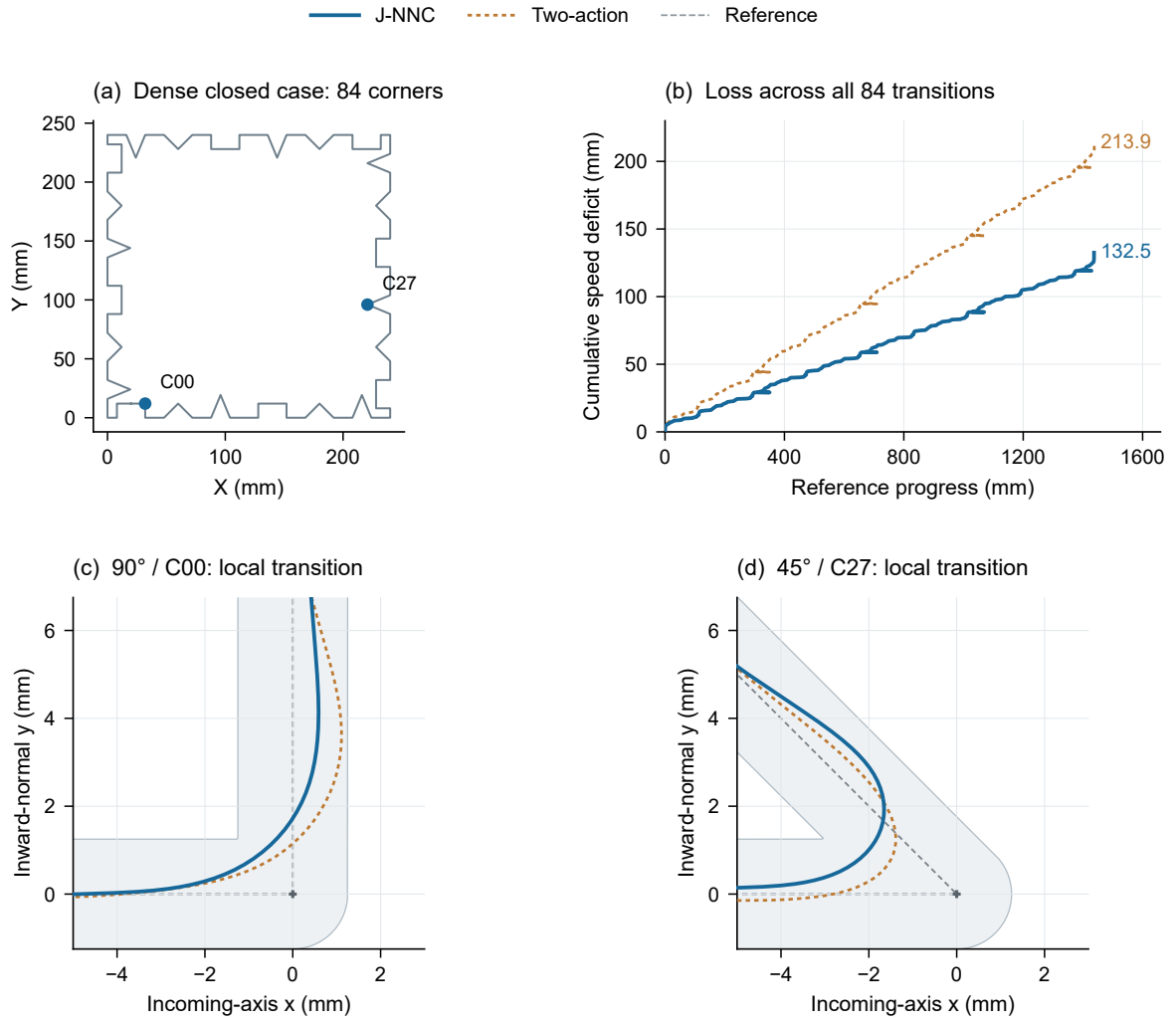



Fig. 9: Comparison with two-action control at 1 ms: (a) dense closed path and locations of the local comparisons; (b) cumulative speed deficit over the complete execution, including all 84 corners and stopping; (c,d) executed 90° and 45° transitions. The curves correspond to the controllers in Tab. 1.

The local views in Fig. 9(c,d) show how the controllers use the tolerance band. J-NNC achieved 4.368 s of near-limit cruising on the dense path, compared with 0.114 s for the selected two-action controller. Total speed deficit decreased from 213.867 to 132.502 mm. Of the 1.036 s saved, 0.814 s came from lower speed deficit and 0.222 s from shorter travel. Both shorter travel and lower speed deficit contributed to the time saving. An equal-budget PPO comparison is reported in Appendix B.

Table 3 reports execution times by model ID. All listed models satisfied the motion constraints, completed both paths and met the speed-deficit criterion at all 34 long-path corners, with dense-case gains of 5.94%–6.07%. All main figures use model 923202.

Tab. 3: Complete execution times by model ID at 1 ms. All satisfy the physical constraints, completion requirements and all-corner criteria. Dense-case gains are relative to the fastest valid two-action time of 15.798 s.

| Model ID | Long path (s) | Dense case (s) | Dense gain (%) |
|---|---|---|---|
| 923201 | 444.178 | 14.859 | 5.94 |
| 923202 (main) | 444.177 | 14.839 | 6.07 |
| 923203 | 444.178 | 14.839 | 6.07 |

The long-path results show sustained cruising and no increase in corner speed deficit relative to either baseline. The dense path, where consecutive corner transitions occupy a larger share of the route, showed a larger time saving.

## 5. Conclusion

This paper presented J-NNC for CNC motion planning along mixed CAM-generated paths under contour and kinematic constraints. The policy jointly selects steering, feedrate and Adaptive Preview to coordinate trajectory shaping and feedrate regulation. The planar KCM projects the combined request into motion satisfying the velocity, acceleration and jerk limits. The policy uses 14 current geometric and kinematic inputs without stored history.

The same selected model completed the 44.420514 m long path and the 84-corner dense path, including ordered traversal, braking and rest. Speed deficit at all 34 long-path corners was no greater than for two-step planning or trained two-action control; complete execution time was 444.177 s. Dense-path execution took 14.839 s, reducing time by 6.07% relative to the fastest valid two-action result and by 25.65% relative to two-step planning. The local trajectories and V/A/J profiles show turning before the corner, feedrate recovery and sustained near-limit cruising within the tolerance band.

Future work will examine servo lag, axis-level constraints and machining performance.

## Appendix A. Kinematic Constraint Module

J-NNC and the two-action controller use the same KCM. At interpolation step $k$, the module selects a feasible next velocity $w$ from the execution layer's velocity request. Given the current velocity $v_k$, acceleration $a_k$ and interval $h$, the V/A/J constraints define the feasible set

$$\mathcal{C}_k = B(0, 100) \cap B(v_k, 1998h) \cap B(v_k + ha_k, 62437.5h^2), \tag{A.1}$$

where $B(c, r)$ is a closed disk with center $c$ and radius $r$. The three disks constrain speed, velocity change and acceleration change, respectively. Internal acceleration and jerk limits leave margins below 2000 mm/s$^2$ and 62500 mm/s$^3$. Candidate enumeration projects the velocity request onto their intersection.

A release check also requires a feasible subsequent reduction of acceleration along a straight line or constant-speed turn. For successive velocity vectors of magnitude $r > 0$, let $d$ and $d'$ be consecutive angular increments. If their signs agree and $|d'| \leq |d|$, next-step jerk $j_{\text{next}}$ satisfies

$$\|j_{\text{next}}\|_2 \leq \frac{\|a\|_2^2}{r} + \frac{r|d' - d|}{h^2}. \tag{A.2}$$

For the release-check jerk bound $B = 62375$ mm/s$^3$ and $\|a\|_2^2/r < B$, the angular increment is reduced by

$$|d| - |d'| = \min\left\{|d|, \frac{h^2}{2r}\left(B - \frac{\|a\|_2^2}{r}\right)\right\}. \tag{A.3}$$

Each candidate is checked against the internal constraints. When speed already equals the requested 100 mm/s, projection includes feasible points on the constant-speed circle and selects the point closest to the velocity request. Otherwise, disk projection and a verified fallback determine the next state.

Algorithm 1 summarizes the execution procedure.

**Algorithm 1** Complete execution with KCM constraints

**Require:** Ordered reference and current $(x_k, v_k, a_k)$

**while** reference coverage is incomplete and no failure has occurred **do**
    At each decision update, encode the 14 current inputs and evaluate the policy
    Decode $(\alpha, F, L)$ and hold for 20 ms
    **for** each integration step before the next decision or termination **do**
        Update guide direction and filtered desired velocity
        Form the bounded jerk and next-velocity requests
        Construct the three disks in Eq. (A.1)
        Project the request and check straight-line or constant-speed acceleration release
        If necessary, search from a verified feasible fallback state
        Set $a_{k+1} = (v_{k+1} - v_k)/h$ and $x_{k+1} = x_k + hv_{k+1}$
        Update ordered coverage and check geometric feasibility
    **end for**
**end while**
**if** reference coverage is complete and no failure has occurred **then**
    Execute nonreversing braking and at least 20 ms rest
**end if**
Check geometry throughout each command interval and reconstruct vector motion from planar positions

## Appendix B. Policy Representation and Learning Settings

### *Appendix B.1. Policy Representation*

The base actor $m_\theta(s)$ has layers 14–64–64–3 with hyperbolic tangent (tanh) hidden activations; the critic has layers 14–64–64–1. The preview and joint-action terms use

static feature vectors $\phi^L(s)$ and $\phi^J(s)$ with 96 and 192 components, respectively. Centers $c_i$, positive coordinate scales $\sigma$ and support radii $r_i$ define

$$d_i = \|(s - c_i)/\sigma\|_2 / r_i, \qquad \phi_i(s) = \max(1 - d_i, 0)^4 (4d_i + 1). \tag{B.1}$$

With preview coefficient row $w_L$, joint coefficient matrix $W$, and $e_3 = (0, 0, 1)^\mathsf{T}$, deterministic execution uses

$$\begin{aligned} \mu_\theta(s) &= m_\theta(s) + 0.4 e_3 \tanh(w_L \phi^L(s)) \\ &\quad + \operatorname{diag}(0.01, 0.08, 0.4) \tanh(W \phi^J(s)), \\ u &= \operatorname{clip}_{[-c,c]}(\mu_\theta(s)), \qquad c = 1 - 10^{-10}. \end{aligned} \tag{B.2}$$

Clipping precedes decoding in Eq. (9), with a small margin to avoid numerical endpoints. The actor contains 5987 fitted network and output coefficients. Both feature banks use only the current 14 inputs, with one shared radius per bank. PPO samples a diagonal Gaussian centered on $\mu_\theta$; execution uses its mean.

*Appendix B.2. Learning Procedure*

Learning combines PPO with fitting to feasible actions recorded from an earlier policy. The four PPO stages contain 16, 24, 24 and 24 updates, interleaved with action fitting. The final stage uses Eq. (10) and batches of 1536 long-path and 512 dense-path decisions; earlier stages also include speed deficit.

The critic fits undiscounted returns and bootstraps from its value estimate at the end of unfinished segments. Each actor update combines the clipped PPO loss with a recorded-action fitting loss (Tab. B.1), recomputes advantages and applies Kullback–Leibler (KL) backtracking.

Tab. B.1: Parameters of the last PPO training stage.

| Setting | Value |
|---|---|
| Actor proposal learning rate / critic rate | $3 \times 10^{-4}$ / $10^{-3}$ |
| PPO ratio clip / entropy coefficient | 0.2 / 0.001 |
| Discount / advantage parameter | 1 / 0.98 |
| Actor epochs / critic epochs | 6 / 32 |
| Minibatch / fresh batch decisions | 256 / 2048 |
| Actor / critic gradient norm limits | 0.5 / 1.0 |
| Post-update divergence limit | 0.03 |
| Auxiliary action-fit weight | 0.5 |
| Auxiliary coordinate scales | $(0.015, 0.15, 0.3)$ |

Final fitting holds feature centers and radii fixed and uses 12000 adaptive moment estimation (Adam) iterations. The learning rate starts at $2 \times 10^{-4}$ and is multiplied by 0.3 at iterations 4000 and 8000. Each batch contains 256 records per case; loss scales are $(0.015, 0.15, 0.3)$ and the gradient limit is 1.

*Appendix B.3. Comparison Settings*

The two-action actor has layers 14–64–64–2, 192 joint features and 5634 coefficients. It uses the first two correction scales and identical clipping. Fixed preview $L = 3$ mm was selected from $\{1.5, 3, 4.5, 6, 8, 12\}$ mm before fitting and PPO training. The controllers use different training schedules and budgets.

The models in Table 3 share recorded-action data and use the same static-feature formulation, with model-specific refinement settings.

A separate matched PPO comparison starts with identical steering/feedrate requests and constant preview. Each arm receives 48 updates and 98304 new decisions; updates 0, 24 and 48 are evaluated on both paths and interpolation intervals. Successful checkpoints are ranked by worst normalized time, then mean normalized time and update index; the initial model is retained if no improvement occurs. Action standard deviations are $(0.002, 0.015, 0.02)$ and the action-fit weight is 0.25. Execution-time differences remain within the numerical sensitivity margin.

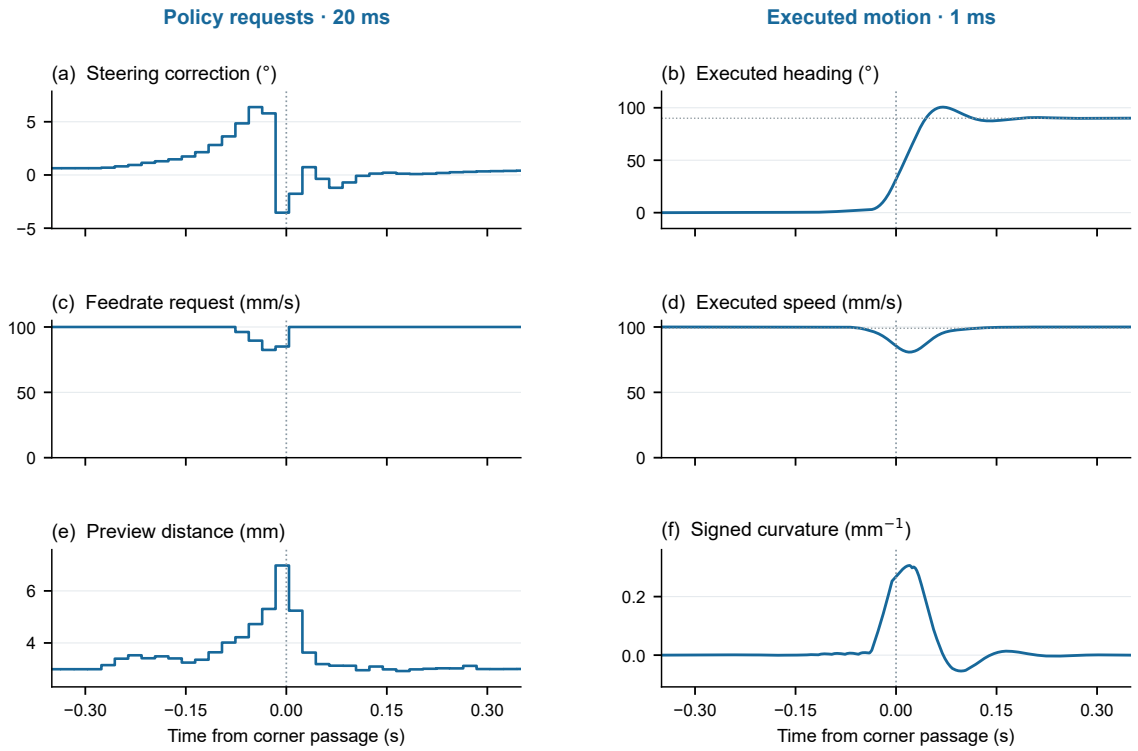



Fig. B.1: Actions and executed motion at the 90°/C02 corner in Fig. 6. Left: steering correction, feedrate request and preview distance held between 20 ms updates. Right: heading, speed and signed curvature from 1 ms positions. Time $t = 0$ marks the first sample reaching the corner in ordered progress. Heading is relative to the incoming tangent; steering, heading and curvature are positive inward. Dotted lines mark the outgoing 90° heading and 99 mm/s cruising threshold. Signals are unsmoothed and share the time axis.

## Appendix C. Execution-Resolution Check

Evaluations were repeated at $h = 0.5$ ms with unchanged policy parameters, physical limits and 20 ms decisions (Table C.1). All listed constrained controllers remained feasible. Maximum J-NNC contour deviation was 1.249077 mm on the dense path, within the 1.25 mm tolerance.

Let $\delta T$ be the absolute execution-time change between 1 and 0.5 ms. The numerical sensitivity criterion requires gains at both intervals to exceed $\max\{0.100, 5(\delta T_{\text{J-NNC}} + \delta T_{\text{baseline}})\}$ seconds, where the subscripts identify the compared controllers. Long-path gains were 1.008/0.990 s over two-step planning and 0.537/0.536 s over two-action control, exceeding margins of 0.100 and 0.175 s. Dense-path gains over the fastest valid two-action times (15.798/15.8755 s) were 0.959/0.997 s, exceeding the 0.585 s margin.

Tab. C.1: Complete execution times at 0.5 ms. The primary comparison at 1 ms is given in Tab. 1.

| Case | J-NNC (s) | Two-action (s) | Two-step planner (s) |
|---|---|---|---|
| Long path | 444.1950 | 444.7310 | 445.1850 |
| Dense case | 14.8785 | 15.8955 | 19.9565 |

The all-34-corner result held at both intervals with wider ±20 and ±30 mm windows. At 0.5 ms, models 923201, 923202 and 923203 completed the long path in 444.1955, 444.1950 and 444.1955 s, and the dense path in 14.8980, 14.8785 and 14.8775 s. All met feasibility and corner criteria; dense-path gains over the fastest valid two-action result were 6.16%–6.29%.

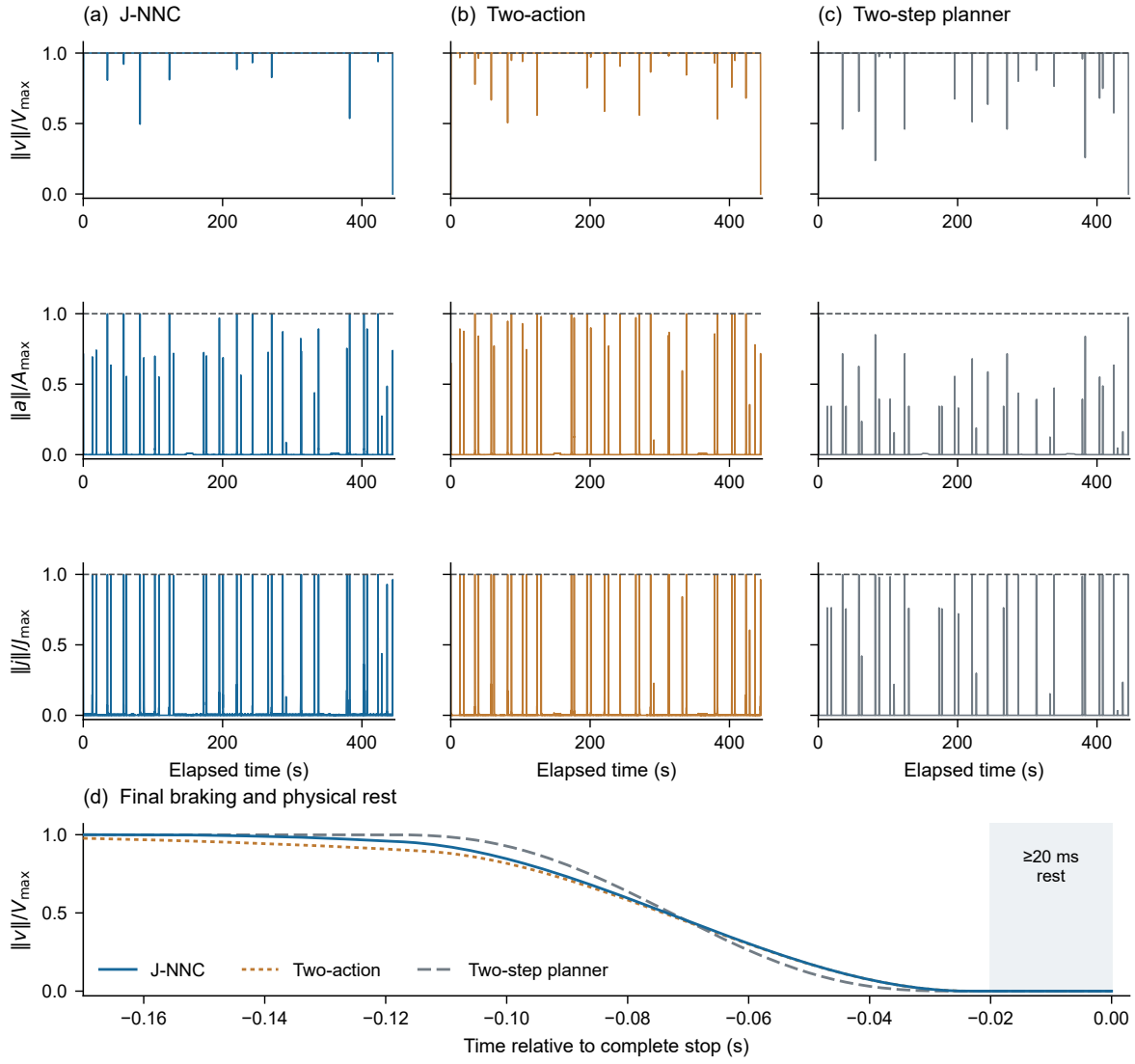



Fig. C.1: Complete long-path profiles at 1 ms. Columns: J-NNC, two-action control and two-step planning. Rows: normalized vector speed, acceleration and jerk. Display reduction preserves extrema; the lower panel includes braking and at least 20 ms of rest.

## Declaration of generative artificial intelligence use

During manuscript preparation, ChatGPT (OpenAI) was used for language editing and translation under the authors' direction. The authors retain full responsibility for the content of the manuscript.

## References


[1] Yuwen Sun, Jinjie Jia, Jinting Xu, Mansen Chen, and Jinbo Niu. Path, feedrate and trajectory planning for free-form surface machining: A state-of-the-art review. *Chinese Journal of Aeronautics*, 35(8):12–29, 2022.

[2] Guangwen Yan, Desheng Zhang, Jinting Xu, and Yuwen Sun. Corner smoothing for CNC machining of linear tool path: A review. *Journal of Advanced Manufacturing Science and Technology*, 3(2):2023001, 2023.

[3] Kaan Erkorkmaz and Yusuf Altintas. High speed CNC system design. part I: jerk limited trajectory generation and quintic spline interpolation. *International Journal of Machine Tools and Manufacture*, 41(9):1323–1345, 2001.

[4] A. C. Lee, M. T. Lin, Y. R. Pan, and W. Y. Lin. The feedrate scheduling of NURBS interpolator for CNC machine tools. *Computer-Aided Design*, 43(6):612–628, 2011.

[5] Burak Sencer, Kosuke Ishizaki, and Eiji Shamoto. High speed cornering strategy with confined contour error and vibration suppression for CNC machine tools. *CIRP Annals*, 64(1):369–372, 2015.

[6] S. Tajima and B. Sencer. Kinematic corner smoothing for high speed machine tools. *International Journal of Machine Tools and Manufacture*, 108:27–43, 2016.

[7] Yong Zhang, Hui Zhang, Jiadong Shi, Jiali Jiang, and Mingyong Zhao. Development of an adaptive-feedrate planning and iterative interpolator for parametric toolpath with normal jerk constraint. *International Journal of Precision Engineering and Manufacturing*, 22:73–81, 2021.

[8] Yifei Hu, Xin Jiang, Guanying Huo, Cheng Su, Hexiong Li, Li-Yong Shen, and Zhiming Zheng. Enhancing five-axis CNC toolpath smoothing: Overlap elimination with asymmetrical B-splines. *CIRP Journal of Manufacturing Science and Technology*, 52:36–57, 2024.

[9] W. Fan, C. Lee, and J. Chen. A realtime curvature-smooth interpolation scheme and motion planning for CNC machining of short line segments. *International Journal of Machine Tools and Manufacture*, 96:27–46, 2015.

[10] Y. Zhang, P. Ye, J. Wu, and H. Zhang. An optimal curvature-smooth transition algorithm with axis jerk limitations along linear segments. *The International Journal of Advanced Manufacturing Technology*, 95:875–888, 2018.

[11] De-Ning Song, Jian-Wei Ma, Yu-Guang Zhong, and Jian-Jun Yao. Global smoothing of short line segment toolpaths by control-point-assigning-based geometric smoothing and FIR filtering-based motion smoothing. *Mechanical Systems and Signal Processing*, 160:107908, 2021.

[12] Z. Yang, L. Shen, C. Yuan, and X. Gao. Curve fitting and optimal interpolation for CNC machining under confined error using quadratic B-splines. *Computer-Aided Design*, 66:62–72, 2015.

[13] Bingcai Wu, Junyan Ma, Lutao Wei, Xiaoping Liao, and Juan Lu. NURBS interpolator with scheduling scheme combining cubic and quartic S-shaped feedrate profiles under drive and chord error constraints. *Computer-Aided Design*, 152:103380, 2022.

[14] M. Tsai, H. Nien, and H. Yau. Development of a real-time look-ahead interpolation methodology with spline-fitting technique for high-speed machining. *The International Journal of Advanced Manufacturing Technology*, 47:621–638, 2010.

[15] De-Ning Song, Yu-Guang Zhong, and Jian-Wei Ma. Look-ahead-window-based interval adaptive feedrate scheduling for long five-axis spline toolpaths under axial drive constraints. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 234(13):1656–1670, 2020.

[16] Huan Zhao, Limin Zhu, and Han Ding. A real-time look-ahead interpolation methodology with curvature-continuous B-spline transition scheme for CNC machining of short line segments. *International Journal of Machine Tools and Manufacture*, 65:88–98, 2013.

[17] M. Tsai and Y. Huang. A novel integrated dynamic acceleration/deceleration interpolation algorithm for a CNC controller. *The International Journal of Advanced Manufacturing Technology*, 87:279–292, 2016.

[18] Molong Duan and Chinedum Okwudire. Minimum-time cornering for CNC machines using an optimal control method with NURBS parameterization. *The International Journal of Advanced Manufacturing Technology*, 85:1405–1418, 2016.

[19] B. Li, H. Zhang, and P. Ye. Driving force constraint corner smoothing method for high-speed machine tools. In *ASME 2017 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, page V006T10A012, 2017.

[20] Hexiong Li, Xin Jiang, Guanying Huo, Cheng Su, Shiwei Zhou, Bolun Wang, Yifei Hu, and Zhiming Zheng. An integrated trajectory smoothing method for lines and arcs mixed toolpath based on motion overlapping strategy. *Journal of Manufacturing Processes*, 95:242–265, 2023.

[21] S. Sun, P. Zhao, T. Zhang, B. Li, and D. Yu. Smoothing interpolation of five-axis tool path with less feedrate fluctuation and higher computation efficiency. *Journal of Manufacturing Processes*, 109:669–693, 2024.

[22] Bingran Li, Hui Zhang, Peiqing Ye, and Jinsong Wang. Trajectory smoothing method using reinforcement learning for computer numerical control machine tools. *Robotics and Computer-Integrated Manufacturing*, 61:101847, 2020.

[23] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. arXiv:1707.06347.

[24] Burak Sencer, Kosuke Ishizaki, and Eiji Shamoto. A curvature optimal sharp corner smoothing algorithm for high-speed feed motion generation of NC systems along linear tool paths. *The International Journal of Advanced Manufacturing Technology*, 76(9–12):1977–1992, 2015.